
A VOCATIONAL TRAINING PROJECT REPORT

In TechNet Vocational Training Pvt Ltd

PROJECT ON "TOC WORKSPACE"

Interactive Theory of Computation Learning and Simulation Platform

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in **COMPUTER SCIENCE / INFORMATION TECHNOLOGY**

Submitted by:

Ayush Nashine

University Roll No. : 301402223048

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

[SHRI SHANKARACHARYA TECHINCAL CAMPUS]

[Bhilai ,Chhattisgarh]

JULY 2026

CERTIFICATE

This is to certify that the vocational training project report entitled “ **TOC WORKSPACE** ” : Interactive Theory of Computation Learning and Simulation Platform" is a bona fide record of the work carried out by **AYUSH NASHINE** , University Roll No. **301402223048** , a student of Bachelor of Technology in Computer Science / Information Technology, under my supervision and guidance during the academic year 2025-2026.

The work presented in this report has not been submitted elsewhere, either in part or in full, for the award of any other degree or diploma of this or any other institute or university. The project embodies the original work of the student and, to the best of my knowledge, represents a satisfactory standard of quality suitable for the award of the degree.

I wish the candidate every success in future endeavours.

Date: _____

Place: _____

_____ [Faculty/Mentor Name] [Designation]
Department of Computer Science & Engineering [Name of Institute / University]

ACKNOWLEDGEMENT

The successful completion of this vocational training project would not have been possible without the guidance, support, and encouragement of several individuals, to whom I express my sincere and heartfelt gratitude.

I would like to extend my profound thanks to my project guide and mentor, [Faculty/Mentor Name], [Designation], Department of Computer Science & Engineering, for their invaluable guidance, constant supervision, insightful suggestions, and constructive feedback throughout the duration of this project. Their expertise in the domain of Theory of Computation and web technologies greatly enriched the quality and depth of this work.

I am also thankful to the Head of the Department and all the faculty members of the Department of Computer Science & Engineering for providing the necessary academic environment, resources, and infrastructure that facilitated the smooth execution of this project.

I would like to acknowledge the continuous support of my classmates and peers, whose feedback during testing of the TOC Workspace platform helped identify usability issues and improve the overall design of the application.

Finally, I extend my deepest gratitude to my family and friends for their unwavering encouragement, patience, and moral support throughout the course of this vocational training and the preparation of this report.

[Student Name]

ABSTRACT

Theory of Computation (TOC) is a foundational subject in Computer Science and Engineering curricula that deals with formal languages, automata theory, computability, and the theoretical limits of computation. Traditionally, students learn TOC concepts such as Deterministic Finite Automata (DFA), Non-Deterministic Finite Automata (NFA), Pushdown Automata (PDA), and Turing Machines through hand-drawn state diagrams and manual trace tables, an approach that is time-consuming, error-prone, and offers no immediate feedback on the correctness of a constructed automaton.

This report presents the design, development, and evaluation of "TOC Workspace", a browser-based, interactive learning and simulation platform that enables students and instructors to visually construct, simulate, and validate automata and formal grammars in real time. The platform is built entirely using client-side web technologies — HTML5, CSS3, and JavaScript (ES6) — with JSON used as the underlying data-interchange and persistence format, and is deployed through Netlify's continuous-deployment and static hosting pipeline.

TOC Workspace provides dedicated modules for DFA, NFA and ϵ -NFA, Mealy and Moore machines, Pushdown Automata, and single-tape as well as multi-tape and non-deterministic Turing Machines. It further includes a Context-Free Grammar (CFG) Studio supporting grammar simplification and parse tree visualization, an automatic Regular Expression to Finite Automaton converter, and a Grammar to Non-deterministic Pushdown Automaton (NPDA) conversion utility. Every automaton created within the workspace can be exported to, and imported from, a structured JSON representation, allowing models to be saved, shared, version-controlled, and reloaded across sessions.

To reinforce learning, the platform incorporates an assignment and quiz module that allows learners to attempt auto-graded exercises on formal language concepts, along with an Admin/Architect review portal that allows instructors to review submitted automata, monitor learner progress, and curate the exercise repository. The application follows a modular, component-based architecture, uses a finite-state simulation engine

capable of animating string acceptance step by step, and stores all application state in the browser using JSON structures, eliminating the need for a dedicated backend server or database.

The resulting system demonstrates that complex theoretical computer science concepts can be made significantly more accessible and engaging through carefully designed interactive visualization tools, without incurring the operational overhead of server-side infrastructure. This report documents the complete lifecycle of the project — from problem identification and requirement analysis to system design, implementation, testing, and future scope — and evaluates the platform's effectiveness as a pedagogical aid for undergraduate Theory of Computation courses.

TABLE OF CONTENTS

CERTIFICATE.....	2
ACKNOWLEDGEMENT.....	3
ABSTRACT.....	4
TABLE OF CONTENTS	6
1. INTRODUCTION TO THEORY OF COMPUTATION	10
2. PROBLEM STATEMENT.....	12
3. OBJECTIVES.....	13
4. LITERATURE REVIEW.....	15
4.1 Textbook-Based Foundations	15
4.2 Existing Automata Simulation Tools.....	15
4.3 Gaps Identified.....	16
5. EXISTING SYSTEM	17
5.1 Manual, Pen-and-Paper Methods	17
5.2 Desktop Simulation Software (e.g., JFLAP).....	17
5.3 Narrow-Scope Web Simulators.....	17
5.4 Limitations of the Existing System.....	18
6. PROPOSED SYSTEM	19
6.1 Unified, Modular Workspace.....	19
6.2 Zero-Installation, Browser-Native Delivery	19

6.3 JSON as a Universal, Portable Data Format	19
6.4 Step-by-Step Visual Simulation	20
6.5 Integrated Assessment and Review	20
7. SOFTWARE AND HARDWARE REQUIREMENTS	21
7.1 Software Requirements	21
7.2 Hardware Requirements.....	22
8. TECHNOLOGIES USED.....	23
8.1 HTML5	23
8.2 CSS3	23
8.3 JavaScript (ES6+)	23
8.4 JSON (JavaScript Object Notation)	23
8.5 Netlify	24
8.6 Supporting Tools.....	24
9. SYSTEM ARCHITECTURE	25
9.1 Presentation Layer	25
9.2 Application / Logic Layer	25
9.3 Data / Persistence Layer	25
9.4 Deployment Layer	25
9.5 Architectural Diagram (Textual Representation).....	26

10. MODULES DESCRIPTION	27
10.1 DFA Module.....	28
10.2 NFA / ϵ -NFA Module	28
10.3 Mealy and Moore Machine Module.....	28
10.4 Pushdown Automata (PDA) Module	29
10.5 Turing Machine Module	29
10.6 Context-Free Grammar (CFG) Studio	29
10.7 Regular Expression to Automaton Converter.....	30
10.8 Grammar to NPDA Converter	30
10.9 JSON Import/Export Utility	30
10.10 Assignment and Quiz Module.....	30
10.11 Admin/Architect Review Portal	31
11. WORKFLOW OF THE APPLICATION	32
11.1 Learner Workflow.....	32
11.2 Instructor Workflow	32
11.3 Conversion Utility Workflow (Illustrative Example)	33
12. ALGORITHMS USED.....	34
12.1 DFA/NFA Simulation Algorithm	34
12.2 Subset Construction (NFA $\rightarrow$ DFA).....	34

12.3 Thompson's Construction (Regex $\rightarrow$ NFA).....	34
12.4 PDA Simulation Algorithm.....	35
12.5 Turing Machine Simulation Algorithm	35
12.6 CFG Parsing and Parse-Tree Construction	35
12.7 Grammar Simplification Algorithms	36
12.8 Grammar-to-NPDA Construction.....	36
13. DATABASE / JSON STORAGE DESIGN	37
13.1 JSON Schema for an Automaton	37
13.2 JSON Schema for a Pushdown Automaton	37
13.3 JSON Schema for a Turing Machine.....	37
13.4 JSON Schema for a Context-Free Grammar	38
13.5 Storage Strategy	38
13.6 Rationale for a JSON-Based Design.....	38
14. ADVANTAGES.....	39
15. LIMITATIONS	40
16. FUTURE ENHANCEMENTS.....	41
17. CONCLUSION.....	42

1. INTRODUCTION TO THEORY OF COMPUTATION

Theory of Computation (TOC) is one of the core theoretical pillars of Computer Science, concerned with understanding what problems can be solved by computational devices, how efficiently they can be solved, and what formal models best describe the behaviour of computing machines. The subject is traditionally divided into three broad areas: Automata Theory and Formal Languages, Computability Theory, and Complexity Theory. At the undergraduate level, TOC is most commonly taught through Automata Theory, which studies abstract machines — such as Finite Automata, Pushdown Automata, and Turing Machines — and the classes of languages they recognize.

Finite Automata (both deterministic and non-deterministic) recognize regular languages and form the theoretical foundation for lexical analyzers, text-processing utilities, and pattern-matching engines used throughout software engineering. Pushdown Automata extend finite automata with an auxiliary stack and recognize context-free languages, which underlie the syntax of virtually all modern programming languages and are central to parser design in compiler construction. Turing Machines, the most powerful model discussed in an undergraduate curriculum, formalize the very notion of an algorithm and are used to reason about decidability, undecidability, and the limits of what can be computed at all.

Despite its practical relevance to compiler design, natural language processing, and digital circuit design, TOC is frequently perceived by students as one of the most abstract and difficult subjects in the computer science curriculum. This difficulty largely stems from the fact that the subject is taught primarily through pen-and-paper notation — state diagrams drawn on a whiteboard, transition tables copied into notebooks, and manual derivations that are tedious to verify by hand. Students rarely get the opportunity to test whether the automaton they have constructed actually behaves correctly on a range of input strings, and errors in a constructed machine often go undetected until an examination.

"TOC Workspace " was conceived to bridge this gap between theoretical instruction and practical, hands-on experimentation. It is a purely client-side, browser-based application that gives learners an interactive canvas on which finite automata, pushdown automata, Turing machines, Mealy/Moore machines, and context-free grammars can be constructed, edited, simulated, and validated instantly. By visualizing the execution of an automaton on a given input string step by step, the platform converts an otherwise abstract and static notation into a dynamic, observable process, thereby significantly improving comprehension and retention of TOC concepts.

2. PROBLEM STATEMENT

Undergraduate students studying Theory of Computation frequently struggle to internalize the operational semantics of abstract machines such as DFA, NFA, PDA, and Turing Machines because conventional teaching methods rely almost entirely on static diagrams and manual trace tables. This creates several recurring difficulties:

- Students cannot verify, on their own, whether a hand-drawn automaton correctly accepts or rejects a given set of strings until it is manually traced or checked by an instructor.
- Constructing and debugging automata by hand is slow, and mistakes in state transitions are difficult to spot visually on paper.
- There is no convenient, standard way for students to save, share, or reuse automata they have designed across different exercises or study sessions.
- Concepts such as non-determinism, epsilon transitions, stack operations in a PDA, and tape movement in a Turing Machine are inherently dynamic processes that are poorly conveyed by static, non-interactive representations.
- Existing computer-based TOC simulators are often either prohibitively complex for a beginner, restricted to a single automaton type, require desktop installation, or lack a unified workspace covering the full breadth of a typical TOC syllabus, including grammars and regular expressions.
- Instructors lack a lightweight, easily deployable tool to assign, collect, and review automaton-construction exercises without relying on heavyweight learning-management infrastructure.

The central problem addressed by this project, therefore, is the absence of a single, accessible, browser-based platform that allows learners to interactively construct, simulate, validate, import/export, and be assessed on the full range of formal models covered in a standard Theory of Computation course — without requiring any software installation, server-side infrastructure, or specialized technical setup.

3. OBJECTIVES

The TOC Workspace project was undertaken with the following specific objectives in mind:

1. To design and develop a unified, browser-based workspace that supports the construction and simulation of Deterministic Finite Automata (DFA), Non-Deterministic Finite Automata (NFA) including epsilon-NFA, Mealy and Moore machines, Pushdown Automata (PDA), and Turing Machines (single-tape, multi-tape, and non-deterministic variants).
2. To implement a visual, drag-and-drop state-diagram editor that allows learners to create automata graphically rather than through raw textual notation alone.
3. To build a step-by-step simulation engine capable of animating the execution of an automaton on a user-supplied input string, clearly highlighting the active state(s), stack contents, or tape configuration at every step.
4. To develop a Context-Free Grammar (CFG) Studio that supports grammar entry, simplification, and parse-tree visualization for a given derivation.
5. To implement automated conversion utilities, specifically a Regular Expression to Finite Automaton converter and a Context-Free Grammar to Non-deterministic Pushdown Automaton (NPDA) converter, so that learners can observe the underlying construction algorithms in action.
6. To provide JSON-based import and export functionality for every automaton type, enabling learners to save their work locally, share constructed machines with peers, and reload previously built models.

-
7. To design an assignment and quiz module that allows learners to attempt auto-graded exercises related to formal language theory and receive immediate feedback on their performance.
 8. To build an Admin/Architect review portal through which instructors can review learner-submitted automata, monitor quiz performance, and manage the repository of exercises.
 9. To deploy the completed application on Netlify so that it is accessible to any learner through a web browser, without requiring local installation or server maintenance.

4. LITERATURE REVIEW

A review of existing academic and open-source tools for teaching automata theory and formal languages was conducted prior to designing TOC Workspace, in order to identify the strengths and shortcomings of prevailing approaches.

4.1 Textbook-Based Foundations

The theoretical foundation of this project draws upon the standard treatment of automata theory found in classic textbooks such as Hopcroft, Motwani, and Ullman's "Introduction to Automata Theory, Languages, and Computation" and Michael Sipser's "Introduction to the Theory of Computation". These texts formally define the DFA, NFA, PDA, and Turing Machine models used as the basis for the simulation logic implemented in the platform, including the standard five-tuple and seven-tuple formal definitions and the subset-construction and grammar-to-PDA construction algorithms that the platform automates.

4.2 Existing Automata Simulation Tools

Several desktop and web-based tools already exist for automata visualization. JFLAP (Java Formal Languages and Automata Package) is a widely used academic tool that supports construction and simulation of a broad range of automata and grammars, but it is a Java desktop application requiring local installation of a Java runtime, which limits accessibility on locked-down lab machines and mobile devices. Automaton simulators built with JavaScript, such as various open-source finite-automaton visualizers hosted on GitHub, typically support only a single automaton type (most commonly DFA/NFA) and lack integrated support for pushdown automata, Turing machines, and grammars within the same interface.

Online regular-expression testers and visualizers demonstrate the pedagogical value of converting a textual specification into a graphical automaton, but these tools are

generally scoped narrowly to regular expressions and do not extend to context-free grammars or Turing-computable languages.

4.3 Gaps Identified

The literature and tool survey revealed a consistent gap: no single freely accessible, installation-free tool was found that unifies finite automata, transducers (Mealy/Moore), pushdown automata, Turing machines (including multi-tape/non-deterministic variants), grammar-based tools, and an assessment module within one coherent, JSON-driven workspace. This gap directly motivated the scope and design of TOC Workspace, which consolidates these otherwise fragmented capabilities into a single web application.

5. EXISTING SYSTEM

Prior to the development of TOC Workspace, learners and instructors typically relied on one or a combination of the following approaches to teach and practice Theory of Computation concepts:

5.1 Manual, Pen-and-Paper Methods

The most prevalent approach remains manual construction of state diagrams and transition tables on paper or a whiteboard, followed by manual tracing of input strings step by step. This method offers no automated verification, is highly susceptible to human error, and does not scale to strings of significant length or automata with a large number of states.

5.2 Desktop Simulation Software (e.g., JFLAP)

Tools such as JFLAP provide a graphical interface for constructing and testing a wide range of automata and grammars. However, being a Java desktop application, JFLAP requires a compatible Java Runtime Environment to be installed on every machine on which it is used, results in a dated user interface that has not evolved significantly with modern UX conventions, and offers no built-in mechanism for instructors to assign, collect, and grade exercises.

5.3 Narrow-Scope Web Simulators

A number of independent web-based simulators exist for individual automaton types, most commonly DFA and NFA visualizers built with JavaScript or Python-based web frameworks. While these tools are more accessible than desktop software, they are generally single-purpose, are not designed to interoperate (an automaton built in one tool cannot be exported to another), and rarely extend to context-free grammars, pushdown automata, or Turing machines within the same interface.

5.4 Limitations of the Existing System

- Fragmented tooling: no single tool covers the full breadth of a standard TOC syllabus.
- Installation overhead for desktop tools, which is impractical for quick classroom or take-home use.
- Absence of a standard, portable file format for saving and exchanging automata between tools and users.
- No integrated assessment or instructor-review capability in most simulation tools.
- Dated or inconsistent user interfaces that do not follow modern, responsive web design practices.

6. PROPOSED SYSTEM

TOC Workspace (Toc-Suite) is proposed as a comprehensive, single-page, browser-based application that consolidates the construction, simulation, conversion, storage, and assessment of formal-language models into one coherent workspace. The proposed system directly addresses the limitations identified in the existing system through the following design decisions:

6.1 Unified, Modular Workspace

Rather than building separate tools for each automaton type, TOC Workspace organizes its functionality into clearly separated but visually consistent modules — DFA, NFA/ ϵ -NFA, Mealy/Moore, PDA, Turing Machine, CFG Studio, and Regex-to-Automaton — all accessible from a single navigable interface, allowing a learner to move seamlessly between related concepts (for example, from a regular expression to its equivalent NFA, or from a context-free grammar to its equivalent NPDA).

6.2 Zero-Installation, Browser-Native Delivery

The application is built entirely with HTML5, CSS3, and vanilla JavaScript (ES6) and requires no plug-ins, runtime environments, or installation of any kind. It runs directly in any modern web browser and is hosted as a static site on Netlify, which provides continuous deployment from a source-code repository, global content delivery, and automatic HTTPS, ensuring the application is instantly and reliably accessible to any learner with an internet connection.

6.3 JSON as a Universal, Portable Data Format

Every automaton, grammar, and machine constructed within the workspace is internally represented, and can be exported and imported, as a structured JSON document. This gives learners a lightweight, human-readable, and version-control-friendly format for saving their work, sharing constructed machines with classmates, and submitting completed exercises to instructors, without requiring any database or server-side storage.

6.4 Step-by-Step Visual Simulation

Rather than only reporting whether a string is accepted or rejected, the simulation engine animates each transition step, highlighting the current state (or set of active states for NFAs), the stack contents for PDAs, and the tape/head position for Turing Machines, giving learners direct visual insight into the operational semantics of each computational model.

6.5 Integrated Assessment and Review

The built-in assignment and quiz module allows learners to attempt structured exercises with automatic grading, while the Admin/Architect review portal gives instructors a lightweight dashboard to review learner submissions, monitor aggregate performance, and manage the exercise bank, closing the loop between practice and assessment within the same platform.

7. SOFTWARE AND HARDWARE REQUIREMENTS

This chapter enumerates the software and hardware resources used during the development and deployment of TOC Workspace, as well as the minimum end-user requirements needed to run the application.

7.1 Software Requirements

Category	Requirement
Operating System	Windows 10/11, macOS 12+, Linux (Ubuntu 20.04+) — client machine (development)
Web Browser (client)	Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari (latest two major versions)
Code Editor / IDE	Visual Studio Code (with Live Server / ES6 linting extensions)
Version Control	Git and a remote repository host (e.g., GitHub) for source management
Hosting / Deployment	Netlify (static site hosting with continuous deployment)
Markup & Styling	HTML5, CSS3 (Flexbox/Grid layout)
Scripting Language	JavaScript (ECMAScript 2015/ES6 and above)
Data Format	JSON (JavaScript Object Notation)
Package/Build Tooling	Node.js and npm (for local development tooling, bundling, and linting)
Browser APIs Used	DOM API, Canvas/SVG rendering API, LocalStorage/IndexedDB, File API (import/export)

Table 7.1 — Software Requirements

7.2 Hardware Requirements

Category	Minimum Specification
Processor	Dual-core 1.6 GHz or higher (development machine)
RAM	4 GB minimum (8 GB recommended for development)
Storage	500 MB free disk space for source code, dependencies, and build artefacts
Display	1366 x 768 resolution or higher, for optimal canvas-based diagram editing
Internet Connectivity	Required for initial page load and Netlify deployment; core simulation logic runs client-side
End-User Device	Any device with a modern web browser — desktop, laptop, or tablet

Table 7.2 — Hardware Requirements

8. TECHNOLOGIES USED

TOC Workspace is built entirely on open, standards-based web technologies, deliberately avoiding heavyweight frameworks in order to keep the application lightweight, dependency-free, and easy to deploy as a static site. The key technologies employed are described below.

8.1 HTML5

HTML5 provides the semantic structural backbone of the application. Semantic elements such as `<header>`, `<nav>`, `<main>`, `<section>`, and `<canvas>` are used to organize each module's interface, while the `<canvas>` element (together with SVG for certain diagram overlays) forms the drawing surface on which automaton state diagrams and parse trees are rendered.

8.2 CSS3

CSS3 is used for all visual styling, including the responsive Flexbox and Grid-based layouts that allow the workspace to adapt across desktop and tablet screen sizes, CSS custom properties (variables) for consistent theming across modules, and CSS transitions/animations used to smoothly animate state highlighting during simulation playback.

8.3 JavaScript (ES6+)

JavaScript, using modern ES6+ syntax (classes, arrow functions, destructuring, modules, promises/async-await), implements all of the application's logic — the automaton data models, the simulation engines for each machine type, the drag-and-drop diagram editor, the regex-to-NFA and grammar-to-NPDA conversion algorithms, the parse-tree renderer, JSON import/export handling, and the quiz-grading logic. The use of ES6 modules keeps each automaton type's logic encapsulated in its own file, improving maintainability.

8.4 JSON (JavaScript Object Notation)

JSON serves as the single data-interchange format throughout the application. Every automaton, grammar, and quiz definition is represented internally as a JSON object,

which is used both for in-memory state management and for the import/export feature that allows learners to save and reload their work as .json files.

8.5 Netlify

Netlify is used as the deployment and hosting platform. It provides continuous deployment directly from a Git repository (automatically rebuilding and redeploying the site on every commit), a global content delivery network for fast page loads, automatic HTTPS/SSL, and, optionally, Netlify Functions for any lightweight serverless logic (such as handling quiz-submission webhooks) that may be required in future extensions of the review portal.

8.6 Supporting Tools

- Git and GitHub — version control and collaborative source management.
- Visual Studio Code — primary development environment, with ESLint and Prettier for code-quality enforcement.
- Chrome DevTools — debugging, performance profiling, and responsive-layout testing.
- Node.js/npm — local build tooling and dependency management during development.

9. SYSTEM ARCHITECTURE

TOC Workspace follows a client-centric, modular architecture. Since the application performs all computation — automaton simulation, conversions, and grading — directly within the browser, there is no dependency on a traditional multi-tier server/database backend for its core functionality. The architecture can be described in terms of the following logical layers.

9.1 Presentation Layer

The presentation layer consists of the HTML5/CSS3 user interface, including the navigation shell, module-specific editor canvases, control panels (for adding states, transitions, and running simulations), and result/feedback panels. This layer is responsible for rendering automaton diagrams, animating simulation steps, and displaying quiz and review-portal content.

9.2 Application / Logic Layer

The logic layer, implemented in ES6 JavaScript, contains the core engines of the platform: the automaton data models (DFA, NFA/ ϵ -NFA, Mealy/Moore, PDA, Turing Machine), the simulation engines that execute an automaton on a given input, the conversion algorithms (regex-to-NFA via Thompson's construction, and CFG-to-NPDA construction), the CFG parser and parse-tree builder, and the quiz-evaluation engine. Each module is encapsulated as an independent ES6 class/module to promote separation of concerns and reusability.

9.3 Data / Persistence Layer

In place of a conventional database, the persistence layer is realized through JSON. Automata and quiz data are held in-memory as JavaScript objects during an editing session, can be persisted temporarily to the browser's Local Storage for session continuity, and can be explicitly exported to, or imported from, .json files via the File API for long-term storage, sharing, or submission to the Admin/Architect review portal.

9.4 Deployment Layer

The static assets that comprise the application (HTML, CSS, JS, JSON configuration files) are version-controlled in a Git repository and deployed through Netlify's build-and-deploy pipeline. Netlify serves the compiled static site over a global CDN, and any future serverless logic required for the review portal (such as centralized storage of submissions) can be layered in using Netlify Functions without altering the client-side architecture.

9.5 Architectural Diagram (Textual Representation)

The overall flow of data and control through the system can be represented as follows:

- Learner/Instructor Browser → Presentation Layer (HTML/CSS UI)
- Presentation Layer → Application Logic Layer (ES6 Engines: DFA/NFA/PDA/TM/CFG/Regex/Quiz)
- Application Logic Layer ↔ Data/Persistence Layer (JSON in-memory state, LocalStorage, Import/Export)
- Application (static build) → Netlify CDN → End-User Browser (worldwide access)

Fig. 9.1 — Layered architecture of TOC Workspace (Presentation → Logic → Data → Deployment)

10. MODULES DESCRIPTION

TOC Workspace is organized into eleven principal functional modules, each addressing a distinct concept area of the Theory of Computation syllabus while sharing a common visual and interaction language across the application. Table 10.1 summarizes each module, and the subsections that follow describe them in further detail.

Module	Core Functionality
DFA Module	Visual construction of states/transitions, determinism validation, step-by-step string simulation, minimization utilities.
NFA / ϵ -NFA Module	Construction with multiple transitions per symbol and epsilon moves; subset-construction conversion to equivalent DFA; parallel-path simulation visualization.
Mealy & Moore Machine Module	Construction of transducers with output on transitions (Mealy) or states (Moore); simulation generating output sequences for a given input; inter-conversion between Mealy and Moore forms.
Pushdown Automata (PDA) Module	Stack-augmented transition editor; simulation with live stack visualization; acceptance by final state or empty stack.
Turing Machine Module	Single-tape and multi-tape TM construction; support for non-deterministic TM (NTM) branches; animated tape/head visualization during execution; halting/acceptance/rejection reporting.
CFG Studio	Grammar (production rule) entry and validation; grammar simplification (removal of useless symbols, unit/null productions); derivation and parse-tree visualization.
Regex-to-Automaton Converter	Parses a regular expression and automatically constructs the equivalent NFA using Thompson's construction, with optional subset-construction to DFA.
Grammar-to-NPDA Converter	Automatically derives an equivalent non-deterministic pushdown automaton from a given context-free grammar using the standard top-down construction.
JSON Import/Export Utility	Serializes any automaton/grammar to a structured JSON file and deserializes previously saved JSON files back into the corresponding editor.

Assignment & Quiz Module	Presents structured exercises and objective questions on TOC concepts; auto-grades learner submissions and stores results.
Admin/Architect Review Portal	Instructor-facing dashboard to review learner-submitted automata/quiz attempts, track progress, and manage the exercise repository.

Table 10.1 — Summary of TOC Workspace Modules

10.1 DFA Module

The DFA module allows learners to add states, mark a designated start state and one or more accepting (final) states, and define exactly one outgoing transition per input symbol from each state, enforcing the determinism property intrinsic to DFAs at the point of construction. The module validates completeness of the transition function and flags missing or duplicate transitions. Once constructed, a learner can supply an input string and watch a token move across the diagram from state to state, with the module reporting acceptance or rejection at the end of the string along with the exact sequence of states visited.

10.2 NFA / ϵ -NFA Module

The NFA module relaxes the determinism constraint, permitting multiple transitions on the same symbol from a state as well as epsilon (ϵ) transitions that consume no input. During simulation, the engine tracks the full set of currently active states (rather than a single state), visually highlighting all reachable states after each input symbol, thereby making the concept of non-deterministic branching directly observable. The module also implements the subset-construction algorithm, allowing a learner to convert any constructed NFA into an equivalent DFA at the click of a button.

10.3 Mealy and Moore Machine Module

This module supports the construction of finite-state transducers. In Mealy machines, an output symbol is associated with each transition, whereas in Moore machines, an output symbol is associated with each state. The simulation engine

executes the machine on an input sequence and generates the corresponding output sequence, and the module additionally provides a converter that transforms a Mealy machine into an equivalent Moore machine and vice versa, illustrating the formal equivalence between the two models.

10.4 Pushdown Automata (PDA) Module

The PDA module extends the finite-automaton editor with an auxiliary stack. Each transition may specify a symbol to be popped from the stack and a (possibly empty) sequence of symbols to be pushed. During simulation, the current stack contents are rendered visually alongside the state diagram, updating in real time as the input string is consumed, so that learners can directly observe how the stack governs acceptance of context-free languages such as balanced-parenthesis or palindrome languages.

10.5 Turing Machine Module

The Turing Machine module models an infinite tape (approximated by a dynamically extensible array) together with a read/write head. It supports single-tape and multi-tape configurations, as well as non-deterministic Turing Machines (NTMs) in which a single configuration may branch into multiple successor configurations. The simulation view animates the tape contents and head position at every step, and reports whether the machine halts in an accepting configuration, a rejecting configuration, or does not halt within a bounded number of steps (to guard against infinite loops during interactive use).

10.6 Context-Free Grammar (CFG) Studio

The CFG Studio provides a structured editor for entering context-free grammar production rules. It performs syntactic validation of the entered grammar, offers simplification routines (elimination of unreachable non-terminals, unit productions, and null productions), and, given a start symbol and target string, generates a leftmost derivation which is rendered as an interactive parse tree, allowing learners to visually trace how a string is derived from the grammar's start symbol.

10.7 Regular Expression to Automaton Converter

This utility accepts a regular expression using standard operators (concatenation, union/alternation, Kleene star, and grouping) and automatically constructs the equivalent NFA using Thompson's construction algorithm, rendering the resulting automaton on the shared diagram canvas. Learners may optionally trigger subset construction to view the equivalent minimized DFA, directly linking the algebraic and automaton-based representations of a regular language.

10.8 Grammar to NPDA Converter

Given a context-free grammar, this module automatically constructs an equivalent non-deterministic pushdown automaton using the standard construction that simulates leftmost derivations on the PDA's stack. The resulting NPDA is displayed in the PDA module's canvas and can immediately be simulated on sample strings, allowing learners to connect grammar-based and automaton-based views of context-free languages.

10.9 JSON Import/Export Utility

Every automaton or grammar built in any module can be exported as a self-describing JSON file capturing its states, alphabet, transition function, stack/tape alphabet (where applicable), start and accepting states, and metadata such as the module type and creation timestamp. The same utility allows a previously exported JSON file to be re-imported into the appropriate module, restoring the full diagram and enabling round-trip editing, sharing between learners, and submission of completed exercises.

10.10 Assignment and Quiz Module

This module presents learners with a curated set of exercises — ranging from objective/multiple-choice questions on formal-language theory to hands-on tasks requiring construction of an automaton satisfying a given specification. Objective questions are auto-graded immediately, while construction-based exercises are validated by running the learner's submitted automaton against a bank of test strings and comparing the observed acceptance behaviour with the expected specification.

10.11 Admin/Architect Review Portal

The review portal is a role-gated section of the application intended for instructors. It surfaces learner quiz scores and submitted automata for inspection, allows an instructor to add, edit, or retire exercises in the assignment bank, and provides aggregate statistics (such as average scores per exercise) to help identify concepts that a cohort of learners is finding particularly challenging.

11. WORKFLOW OF THE APPLICATION

This chapter describes the typical end-to-end workflow followed by a learner and, separately, by an instructor while using TOC Workspace.

11.1 Learner Workflow

9. The learner opens the TOC Workspace URL in a web browser; the application loads instantly as a static site served from Netlify's CDN.
10. The learner selects the desired module from the navigation shell (e.g., DFA, NFA, PDA, Turing Machine, CFG Studio, or Regex Converter).
11. Using the visual editor, the learner adds states and transitions (or enters a grammar/regular expression, depending on the module) to construct the desired automaton.
12. The learner enters a test input string and initiates the simulation; the engine animates the automaton's execution step by step and reports the final acceptance/rejection outcome.
13. The learner refines the automaton as needed, repeating steps 3–4 until the automaton behaves as intended on all required test cases.
14. The learner exports the completed automaton to a JSON file for safekeeping, or proceeds directly to the Assignment & Quiz module to attempt a graded exercise.
15. Within the quiz module, the learner answers objective questions and/or submits constructed automata for specific tasks; the platform auto-grades the submission and displays the score and feedback immediately.

11.2 Instructor Workflow

16. The instructor logs into the Admin/Architect review portal using the designated access route.
17. The instructor reviews the current bank of assignments/quizzes and adds, edits, or removes exercises as required for the course schedule.
18. As learners complete exercises, their scores and submitted JSON automata become visible in the review portal's dashboard.

-
19. The instructor inspects individual submissions where deeper review is warranted (for example, borderline or flagged submissions), using the same visual simulation tools available to learners to re-run a submitted automaton against test strings.
 20. The instructor uses the aggregate statistics provided by the portal to identify concepts that require additional in-class coverage.

11.3 Conversion Utility Workflow (Illustrative Example)

As a representative example of the platform's conversion workflow: a learner enters the regular expression $(a|b)^*abb$ in the Regex-to-Automaton module; the engine parses the expression into an abstract syntax tree, applies Thompson's construction to generate an equivalent ϵ -NFA, renders the resulting automaton on the canvas, and offers a one-click subset-construction step to derive and display the equivalent minimized DFA — allowing the learner to directly compare the regular expression, the NFA, and the DFA representations of the same language.

12. ALGORITHMS USED

The correctness and educational value of TOC Workspace rests on a set of well-established formal-language algorithms, each implemented in JavaScript within the application's logic layer. The principal algorithms are outlined below.

12.1 DFA/NFA Simulation Algorithm

For DFA simulation, the engine maintains a single current state and, for each input symbol, looks up the unique transition defined for that state and symbol, advancing to the resulting state or immediately rejecting if no transition is defined. For NFA simulation, the engine maintains a set of current states; for each input symbol, it computes the union of all states reachable via a matching transition from any state in the current set, additionally applying epsilon-closure (repeatedly following ϵ -transitions) before and after each input symbol is consumed. The string is accepted if the final set of active states contains at least one accepting state.

12.2 Subset Construction ($\text{NFA} \rightarrow \text{DFA}$)

The subset-construction algorithm converts a given NFA into an equivalent DFA. Starting from the epsilon-closure of the NFA's start state (treated as the DFA's start state), the algorithm computes, for every reachable subset of NFA states and every input symbol, the epsilon-closure of the union of transitions from that subset, generating a new DFA state for each previously unseen subset. The process terminates when no new subsets are discovered, and any subset containing an NFA accepting state is marked as a DFA accepting state.

12.3 Thompson's Construction ($\text{Regex} \rightarrow \text{NFA}$)

Thompson's construction builds an NFA compositionally from a regular expression's parse tree. Each primitive symbol is represented as a two-state fragment with a single transition; concatenation chains two fragments in sequence; union (alternation) introduces a new start state with epsilon-transitions to both operand fragments, converging to a shared end state; and the Kleene star wraps a fragment with epsilon-transitions that allow it to be skipped entirely or repeated an arbitrary number of times.

The resulting fragment, after processing the entire parse tree, is the complete equivalent NFA.

12.4 PDA Simulation Algorithm

The PDA simulation algorithm extends automaton simulation with an explicit stack data structure. At each step, given the current state, the next input symbol, and the symbol on top of the stack, the engine identifies matching transitions, pops the specified symbol (if any) from the stack, pushes the specified replacement sequence, and moves to the resulting state. Because PDA transitions may be non-deterministic, the engine explores multiple candidate branches (using a bounded-depth search) and reports acceptance if any branch consumes the entire input while reaching an accepting configuration (by final state or empty stack, per the configured acceptance mode).

12.5 Turing Machine Simulation Algorithm

The Turing Machine engine models the tape as a dynamically growing array (extended on demand in either direction) together with a head-position pointer. At each step, the transition function is consulted using the current state and the symbol under the head; the engine writes the specified symbol, moves the head left or right (or keeps it stationary, depending on configuration), and transitions to the resulting state. For multi-tape machines, this process is replicated across each independent tape and head. For non-deterministic Turing Machines, the engine performs a breadth-first exploration of the configuration tree up to a configurable step bound, reporting acceptance if any branch reaches an accepting halting configuration.

12.6 CFG Parsing and Parse-Tree Construction

Given a context-free grammar and a target string, the CFG Studio constructs a leftmost derivation using a recursive descent/backtracking strategy over the grammar's production rules, attempting to match each non-terminal's alternatives against the remaining input. Each successful derivation step is recorded as a parent-child relationship between grammar symbols, which is subsequently rendered as a parse tree using a recursive layout algorithm that positions each node based on the horizontal extent of its subtree.

12.7 Grammar Simplification Algorithms

The CFG Studio additionally implements standard grammar-cleanup algorithms: removal of non-terminals that are unreachable from the start symbol or that cannot ultimately derive any terminal string, elimination of unit productions (rules of the form $A \rightarrow B$, where B is a single non-terminal) by substitution, and elimination of null (ϵ) productions with corresponding adjustment of the remaining rules to preserve the generated language.

12.8 Grammar-to-NPDA Construction

The grammar-to-NPDA converter implements the classical construction that simulates a leftmost derivation using the PDA's stack: the stack is initialized with the grammar's start symbol, and at each step, if the symbol on top of the stack is a non-terminal, it is replaced (non-deterministically, across all matching production rules) by the right-hand side of a production; if it is a terminal, it is matched against, and removed together with, the next input symbol. The string is accepted if the stack (and input) are simultaneously exhausted.

13. DATABASE / JSON STORAGE DESIGN

Because TOC Workspace is designed as a fully client-side application, it does not rely on a conventional relational or NoSQL database server. Instead, all persistent data — constructed automata, grammars, quiz definitions, and submission records — is represented using structured JSON documents, which are held in browser memory during an active session, optionally cached to the browser's LocalStorage for session continuity, and explicitly exported to or imported from standalone .json files for long-term storage and sharing.

13.1 JSON Schema for an Automaton

Every finite-automaton-family object (DFA, NFA, Mealy, Moore) is serialized using a common schema, illustrated below in simplified form:

```
{ "type": "DFA", "states": ["q0", "q1", "q2"], "alphabet": ["0", "1"], "startState":  
"q0", "acceptStates": ["q2"], "transitions": [ { "from": "q0", "symbol": "0", "to":  
"q1" }, { "from": "q0", "symbol": "1", "to": "q0" }, { "from": "q1", "symbol":  
"1", "to": "q2" } ], "metadata": { "createdAt": "2026-06-14T10:15:00Z", "author":  
"learner_id" } }
```

13.2 JSON Schema for a Pushdown Automaton

The PDA schema extends the finite-automaton schema with a stack alphabet and stack operations attached to each transition, capturing the symbol to pop and the sequence of symbols to push, in addition to the input symbol consumed and the resulting state.

13.3 JSON Schema for a Turing Machine

The Turing Machine schema records one or more tape alphabets (for multi-tape configurations), a blank symbol, and a transition table in which each entry specifies the current state and the symbol(s) read across all tapes, mapping to the symbol(s) to write, the head movement direction(s) (left/right/stay) for each tape, and the resulting state; non-deterministic machines permit multiple transition entries for the same (state, symbol) pair.

13.4 JSON Schema for a Context-Free Grammar

A grammar object records the set of non-terminals, the set of terminals, the start symbol, and an ordered list of production rules, each expressed as a non-terminal mapped to one or more alternative right-hand-side symbol sequences.

13.5 Storage Strategy

- In-memory state: the JavaScript object currently being edited in a module, kept in sync with the visual canvas.
- Session persistence: automatic serialization of the in-memory state to browser `LocalStorage` at defined checkpoints, so that a learner's work-in-progress survives accidental page reloads within the same browser.
- Explicit export/import: user-triggered serialization to, and deserialization from, downloadable `.json` files via the browser's File API, used for long-term storage, sharing between users, and quiz submission.
- Quiz and review data: quiz definitions and learner submission records are likewise represented as JSON documents, which the Admin/Architect review portal reads to populate its dashboards; this design keeps open the option of later migrating the review portal's data store to a lightweight cloud database or Netlify Function-backed API without altering the client-side data model.
-

13.6 Rationale for a JSON-Based Design

A JSON-centric storage design was chosen over a traditional database for three principal reasons: it keeps the application entirely static and serverless, which minimizes hosting cost and operational complexity and aligns naturally with Netlify's static-site deployment model; JSON is human-readable and easily inspected, edited, or version-controlled, which is pedagogically valuable since learners can directly examine the formal structure of an automaton they have built; and JSON's structure maps naturally onto the JavaScript object model used throughout the application, avoiding any need for an object-relational mapping layer.

14. ADVANTAGES

- Unified platform covering the full breadth of a typical TOC syllabus — DFA, NFA/ ϵ -NFA, Mealy/Moore machines, PDA, and Turing Machines — within a single, consistent interface.
- Zero installation: runs directly in any modern web browser, removing the setup barriers associated with desktop tools such as JFLAP.
- Immediate, step-by-step visual feedback on automaton behaviour, which significantly improves comprehension of otherwise abstract operational semantics.
- Automated conversion utilities (Regex-to-NFA, Grammar-to-NPDA) that let learners observe canonical construction algorithms in action rather than only reading about them.
- Portable, human-readable JSON format for saving, sharing, and version-controlling automata, without dependency on any proprietary file format.
- Integrated assessment (quiz/assignment module) that closes the loop between practice and evaluation within the same tool.
- Instructor-facing review portal that provides visibility into learner progress without requiring integration with a separate learning-management system.
- Low hosting cost and high availability, owing to Netlify's static-site hosting, global CDN, and continuous-deployment pipeline.
- Lightweight technology stack (HTML5/CSS3/vanilla JavaScript) that keeps the codebase easy to understand, extend, and maintain for future contributors.

15. LIMITATIONS

- Because all computation runs client-side, simulation of very large automata (a large number of states/transitions) or Turing Machines requiring a very high number of steps may be constrained by the performance of the learner's browser and device.
- Non-deterministic Turing Machine and PDA simulations rely on a bounded-depth/step exploration to remain interactive, which means machines that would legitimately run for an extremely large number of steps before halting cannot always be simulated to completion within the browser.
- The current JSON/LocalStorage-based persistence model does not provide centralized, multi-device synchronization of a learner's saved work; a learner must manually export and transfer JSON files between devices.
- The Admin/Architect review portal, in its present form, depends on learners explicitly submitting JSON files or quiz results rather than a fully automated, always-connected submission pipeline, since no dedicated backend database is used.
- The parse-tree visualization for context-free grammars is optimized for moderately sized grammars and derivations; very large or deeply ambiguous grammars can produce parse trees that are visually dense and harder to interpret.
- The platform currently assumes a reasonably modern browser with support for ES6+ JavaScript and Canvas/SVG rendering; legacy browsers are not supported.

16. FUTURE ENHANCEMENTS

- Integration of a lightweight cloud backend (for example, using Netlify Functions together with a managed database) to enable centralized, cross-device storage of learner progress and submissions.
- Support for automaton minimization visualizations (e.g., animated Myhill-Nerode based DFA minimization) to complement the existing construction and simulation features.
- Extension of the Turing Machine module to visualize decidability/undecidability demonstrations, such as simulating a Universal Turing Machine on encoded machine descriptions.
- Real-time collaborative editing, allowing multiple learners to construct or debug an automaton together within the same session.
- Richer analytics in the Admin/Architect review portal, including per-concept mastery tracking and personalized exercise recommendations based on a learner's quiz history.
- Native mobile-optimized touch interactions for the diagram editor, to improve usability on tablets and smartphones.
- Support for additional formal-language constructs, such as linear-bounded automata and Chomsky-hierarchy classification tools, to extend coverage toward advanced/graduate-level TOC topics.
- Accessibility enhancements, including keyboard-only diagram construction and screen-reader-friendly descriptions of automaton state, to broaden the platform's usability.

17. CONCLUSION

The TOC Workspace project set out to address a well-recognized gap in the teaching and learning of Theory of Computation: the absence of a single, accessible, browser-based tool that allows learners to interactively construct, simulate, convert, store, and be assessed on the full range of formal models covered in a standard undergraduate curriculum. Through the design and implementation of a modular, purely client-side application built on HTML5, CSS3, JavaScript (ES6), and JSON, and deployed via Netlify, this project has demonstrated that such a unified platform is both technically feasible and pedagogically valuable.

The resulting system successfully brings together dedicated modules for DFA, NFA/ ϵ -NFA, Mealy and Moore machines, Pushdown Automata, and Turing Machines (including multi-tape and non-deterministic variants), alongside a Context-Free Grammar Studio, automated Regex-to-Automaton and Grammar-to-NPDA converters, a portable JSON-based import/export mechanism, and an integrated assignment/quiz module supported by an instructor-facing review portal. By replacing static, pen-and-paper representations of automata with dynamic, step-by-step visual simulations, the platform makes the operational behaviour of abstract computational models directly observable, which is expected to improve learner comprehension, engagement, and retention of core TOC concepts.

The project also validates the practicality of a lightweight, serverless architecture for educational tooling: by relying entirely on client-side computation and JSON-based data interchange, TOC Workspace avoids the operational overhead of a traditional backend while still delivering a feature-rich, multi-module learning environment. While certain limitations remain — particularly around the scalability of client-side simulation for very large automata and the absence of centralized, cross-device data synchronization — these are well-understood constraints that map onto a clear roadmap of future enhancements, including cloud-backed persistence, collaborative editing, and expanded coverage of advanced formal-language topics.

In summary, this vocational training project has resulted in a functional, extensible, and pedagogically motivated platform that meaningfully advances how Theory of Computation can be taught and practiced in a modern, web-first educational setting, while also providing the student with substantial hands-on experience in front-end web development, algorithm implementation, structured data design, and cloud-based deployment.